

System Architecture Specification

Keepr Storage System

Document Version: 2.7
Date: February 2026
Prepared By: Engineering Team

Document Revision History

Date	Version	Description	Author
Feb 2026	2.7	Initial draft of system architecture and API specifications.	Eng. Team

Contents

1	Overview	2
1.1	Purpose	2
1.2	System Components	2
2	Backend Architecture	3
2.1	Double Clustering Strategy	3
2.1.1	Database Clustering (Neon Postgres)	3
2.1.2	Storage Clustering (Dropbox)	3
2.2	API and Services Specifications	3
3	Frontend Architecture	4
3.1	Framework and Configuration	4
4	Data Management and Security	5
4.1	Storage Implementations	5
4.2	Security Protocol	5

1. Overview

1.1 Purpose

This document outlines the system architecture and technical specifications of the Keepr system. The system provides an advanced, horizontally scalable cloud storage and management solution using a double clustering strategy for database and storage layers.

1.2 System Components

The platform is deployed as a distributed client-server application consisting of three primary layers:

- **Client Application (Frontend):** A Flutter application serving as the primary cross-platform user interface.
- **Application Programming Interface (Backend):** A Node.js service handling request routing, clustering logic, and secure authentication.
- **Storage Layer:** A decentralized configuration utilizing a Dropbox cluster for large binary files (file chunks) and a distributed Postgres SQL cluster (Neon) for application and topology data.

2. Backend Architecture

2.1 Double Clustering Strategy

Keepr's core innovation lies in its dual-layered clustering approach, allowing theoretical infinite scaling by circumventing single-provider limitations.

2.1.1 Database Clustering (Neon Postgres)

The system utilizes a central Master Registry to manage a fleet of Worker Databases.

- **Master Registry:** Defines the topology. The `db_shards` table stores connection credentials and utilization metrics for all worker instances.
- **Dynamic Allocation:** The `dbManager.getFittestDB()` routine queries the Master Registry to identify the optimal worker database where `current_usage < max_capacity`, dynamically load balancing data ingestion.
- **Connection Pooling:** The backend implements intelligent DNS caching and connection pooling to eliminate latency during horizontal shard hops.

2.1.2 Storage Clustering (Dropbox)

To achieve massive storage capabilities without enterprise costs, Keepr clusters multiple Dropbox accounts.

- **Storage Registry:** The `storage_shards` table securely holds OAuth tokens (refresh tokens, app keys, and secrets) for an array of Dropbox accounts.
- **Intelligent Routing:** The `storageManager.getFittestStorageAccount()` algorithm calculates real-time free space, accounting for pending chunk uploads, and routes new file streams to the most available shard.
- **Chunked Uploads:** Large files are dynamically chunked and can be distributed across different storage shards.

2.2 API and Services Specifications

The backend application is developed in JavaScript using the Node.js environment. Key operational services include:

dbManager.js	Responsible for establishing secure, pooled connections to the Master DB and dynamically resolving Worker DBs. It implements retry logic, public DNS fallbacks, and connection string parsing to ensure robust database resilience.
storageManager.js	Handles the lifecycle of storage operations. It features OAuth2 token caching (3.5-hour TTL) to bypass handshake overhead, live space usage validation via Dropbox API, and batch deletion queues with asynchronous job polling.
auth.js	Manages secure user authentication via email-based OTPs leveraging Node-mailer.

3. Frontend Architecture

3.1 Framework and Configuration

The client interface is a Flutter application designed for cross-platform compatibility (Android, Windows, Web).

- **Language:** Dart is utilized for type-safe UI component generation and business logic.
- **Styling:** The UI leverages a modern Glassmorphism design language, defined in `lib/theme/keep_theme.dart` utilizing backdrop filters and translucent layers to create a premium, depth-aware interface.
- **Upload Architecture:** The `folder_upload_service.dart` handles recursive directory walking, allowing users to drop entire folder structures which are then parsed and chunked seamlessly for backend processing.

4. Data Management and Security

4.1 Storage Implementations

- **Relational Database:** PostgreSQL databases manage the application state. The Master Registry DB tracks shards, while the Worker DBs store user metadata, chunks routing info, and file structures.
- **Cluster Storage:** Distributed Dropbox accounts manage the binary payload chunks. The back-end abstracts the Dropbox API to handle batch deletions, chunked session uploads, and asynchronous state polling.

4.2 Security Protocol

- **Environment Isolation:** All sensitive credentials (DB URLs, API secrets) are strictly isolated via environment variables.
- **Token Security:** Refresh tokens are stored securely in the database, with short-lived access tokens cached temporarily in memory.
- **Connection Security:** SSL/TLS is strictly enforced for all Postgres connections (SNI routing support built-in) and Dropbox API requests.